

第 5 章 语音图形界面

融合 GUI 和 VUI 的主要原因有以下 3 个。

第一，视觉通道已不能满足日益复杂的人机交互需要。单纯的视觉通道会使人的信息感知效率受限，例如用户看不到几米外的手机显示什么内容。如果一个空间内存在多个显示屏幕，这时要求用户在短时间内或同一时间内在不同屏幕之间处理大量的视觉信息，就会造成用户的视觉信息过载，导致用户视觉疲劳或遗漏重要的目标信息。

第二，我们的视觉系统和听觉系统是相互依存的，视觉提供的是位于前方的具有丰富细节信息的图像，而听觉提供的则是环绕我们四周的信息。听觉通道有许多视觉通道不具有的优点，例如迫听性、全方位性、变化敏感性、绕射性和穿透性，它们都可以弥补视觉通道的不足，并且降低视觉通道的负载。尤其是在驾驶汽车通过十字路时，在路面和屏幕间来回切换视线是一件既不容易又很危险的事情，所以车载系统的语音交互变得格外重要，关键信息可以通过声音来传达，而且可以通过语音、手势等方式进行交互，这样能有效减少对视觉的需求。

第三，人机交互逐渐从二维平面转向三维空间，包括 VR、AR、智能座舱和跨设备交互，语音交互是最好的隔空交互方式之一。虽然手势、肢体动作、视线追踪等空间交互技术仍在发展中，但是它们和 VR、AR 的 GUI 密切相关，只要将 GUI 和 VUI 提前融合，等这些技术成熟后就可以将它们逐

一融入到未来的设计中，这样就能加速整个人机交互技术的发展。

5.1 GUI和VUI的区别

1962 年，图灵奖得主格拉斯·恩格尔巴特（Douglas Engelbart）发表了一篇论文《提升人类智能：一个概念性的框架》，呈现了依靠技术管理信息，帮助人们互相合作来解决世界经济和环境问题的蓝图。人机交互的各种技术例如 GUI、VUI、AR 和 VR，基本上停留在恩格尔巴特这个理论框架中。但是 GUI 和 VUI 结合在一起需要考虑很多问题，下面首先介绍 GUI 和 VUI 的特点。

5.1.1 GUI的特点

1973 年，施乐 PARC 研究中心推出了世界上第一款拥有图形界面的 Alto 计算机，从此开启了计算机图形界面的新纪元，人机交互正式进入 GUI 时代。GUI 在前期被研究人员定义为以窗口（Windows）、图标（Icons）、菜单（Menu）和指示装置（Pointing Devices）为基础的图形用户界面，也称为 WIMP 界面。在 WIMP 界面里，每个窗口都是一个独立的执行程序，也是执行程序的可视范围。当多个窗口同时出现在桌面（Desktop）上时，用户可以对它们进行交叠和并排的操作。每个图标都能激活一个窗口，不同的图标表示不同的含义，例如文件、文件夹、快捷方式、应用或设备。菜单是可选命令或选项的一个列表。指示装置是指操作界面的设备，鼠标是使用最普遍的，在触摸屏上手指和手写笔也是一种指示装置。常见的 GUI 操作系统如 Windows、OS X、Linux、iOS 和 Android 都属于 WIMP 界面。它们具有以下特点。

1. 通过焦点对看到的界面元素进行直接操作

WIMP 界面能让用户直接操作屏幕上的对象而非通过基于命令的界面

执行命令，这种交互方式被称为“直接操作”。在 GUI 前期，通过鼠标操作界面的问题是鼠标和屏幕之间没有对应关系，所以鼠标的发明人格拉斯·恩格尔巴特在界面里设计了一个具有指向性的小图标来表示鼠标在界面里的当前位置和状态，这个小图标被称为“鼠标指针”，也就是我们常说的光标。通过光标，用户可以对界面里的任何操作对象进行点击、长按、拖曳等操作，这种交互方式被称为“可见即可点”。光标用于显示用户当前的操作对象是谁，这种交互方式被称为“获得焦点”，而触控屏上手指的点击操作也能让窗口、图标或菜单立即获得焦点。总的来说，GUI 的第一个特点是“通过焦点对看到的界面元素进行直接操作”。这句话也意味着 GUI 的第一个缺点是没有焦点的确认，GUI 不知道用户在操作哪个对象。

2. 信息的展现和交互由屏幕和 GUI 元素决定

早期的显示器是不可交互的，所以计算机需要用鼠标和键盘进行操控；当电阻式和电容式触摸屏技术成熟且成本降下来后，更直观、更自然的触觉交互逐渐在手机、平板电脑和穿戴式设备上应用。这些屏幕的大小都是固定的，如果没有用鼠标或用手指触控的方式使界面产生滚动或切换，我们就可以认为屏幕上的内容是固定的，超出屏幕的信息都是不可见的。结合 GUI 的第一个特点，GUI 的第二个缺点是在桌面和窗口中看不到的内容无法被用户阅读和操作。

VR 和 AR 的屏幕离用户的眼睛越来越近，导致用户无法对该屏幕进行触控，因此隔空手势交互成为最重要的交互手段之一。在 VR 和 AR 中，我们可以认为信息已经融入了空间，但是它们的信息展示方式跟手机、电脑并无差异，都依赖于 GUI 的控件、组件和容器，只是展示的形式不同，由于 VR 和 AR 是有深度信息的，所以它的 GUI 元素比基于平面的 GUI 元素复杂。

3. GUI 交互流程是固定的

交互流程由信息架构决定。每个应用的信息架构是固定的，所以 GUI

的交互流程也是固定的。在屏幕较大的 PC 上，Web 一般会有一个导航栏让用户随意切换页面路径，但手机 App 由于体积小的缘故一般是没有导航栏的，所以 GUI 的第三个缺点是不能随时随地切换流程和路径，只能通过实体/虚拟按键或以手势的方式回到特定路径上。

4. 信息会随着不同的排版和布局发生变化

众所周知，中国大陆和欧美一些国家的图书的文字排版都是横排的，阅读方向从左往右，而在日本、韩国，以及中国港澳台地区和古代图书的文字排版都是竖排的，阅读方向从上往下。如果读者不知道文字的阅读方向，是看不懂书里讲的内容的。

在 GUI 的布局设计中，设计师普遍遵循了格式塔原理。格式塔原理是由德国心理学家组成的研究小组试图解释人类视觉的工作原理，它包含的接近性原理、相似性原理、连续性原理、封闭性原理、对称性原理、主体/背景原理和共同命运原理都能对布局设计产生影响，例如互相靠近或相似的物体看起来属于一组、一起运动的物体被感知为属于一组或是彼此相关的。

5. 支持多任务切换

一个窗口代表一项任务，当桌面上有多个窗口被打开时，用户可以通过焦点切换告诉操作系统现在哪个窗口正在被用户使用，这时界面元素会有相应的变化。多任务切换体现出 GUI 的高效率和便捷性。

5.1.2 VUI的特点

在第 3 章已经总结了 VUI 的优势和不足下面结合 GUI 对 VUI 的特点进行介绍。

1. VUI 的交互手段由词组和语法决定

VUI 没有控件、组件、页面等界面元素的说法，所有的交互手段都是由

词组和语法构成的。一般来说，VUI 中的操作对象不是主语就是宾语，但用户说话时不一定会把它们全说出来，因为用户说话时有可能将眼睛看到的、脑海里出现的、上下文提及的事物都默认设定为主语和宾语，说话时可能会忽略它们，缺乏主语/宾语或代词指定不明对于 VUI 来说是一种挑战。

2. VUI 是“所想即所说”

和 GUI 的“可见即可点”不一样，VUI 的特点是“所想即所说”，意思是用户眼睛看到的、脑海里出现的、上下文提及的事物都有可能成为交互对象。所以，GUI 的交互对象仅是当前页面里的界面元素，而 VUI 的交互对象可以是任何事物。

3. VUI 一句话等于一项或多项操作

这是一个体现 VUI 高效率的重要特性。一般来说，GUI 完成一项任务需要分几个步骤，而 VUI 用一句话就可以搞定，例如“帮我设置明天早上 6 点半的闹钟”“我要订一张明天 8 点飞往北京的机票”。但是 VUI 只支持单任务，没有 GUI 的多任务切换的功能，因为 VUI 里的所有信息都需要输入到用户脑海里，用户记不了太多信息，任务的暂且搁置和来回切换会影响用户理解和判断。同时，交互不同任务有着不同的场景，来回切换场景会对语音交互的上下文理解造成很大影响。

4. VUI 交互流程是可变化的

VUI 在流程上没有可视化的分支结构，更没有返回逻辑，这意味着 VUI 每次只能完成一次对话或一个指令，在多轮对话里只能一直向前推进。GUI 的信息架构是固定的，例如在买电影票时要么先选择电影，要么先选择电影院；而 VUI 和 GUI 不太一样，VUI 的信息架构是需要变化的，先选择电影或先选择电影院对于用户来说都是正确的步骤。另外，因为 VUI 没有界面和文字的限制，加上用户的发散性思维，所以用户的行为可能很随意，会随时随地打断当前任务跳到其他流程上，这些都是固定信息架构的 GUI 做不

到的。

5. VUI 不适合用于复杂任务

VUI 没有界面和文字引导,加上用户工作记忆的限制,导致用户接收的信息是有限的。VUI 跟打电话差不多,用户 A 不知道此时此刻电话另一侧的用户 B 在做什么,只能通过对话中的上下文进行内容理解和语境分析,过长的对话会影响用户对内容的理解和判断。因此,VUI 不适合用于复杂任务。

6. VUI 的交互方式由剧本决定

VUI 的剧本跟 GUI 的信息架构不太一样,对于用户而言,说话的内容和风格都会影响用户对语音智能助手性格的认知,所以编写剧本需要考虑是什么样的场景,提前定义好语音智能助手的性格特征,以及怎样和用户对话,包括如何回复用户的问题及如何衔接下一个任务或话题。

7. VUI 可以影响用户情感

GUI 可以通过文案、图形、动效等方式影响用户的情感,而声音更能影响用户的情感,所以语音智能助手说的一字一句都有可能影响用户的感受,例如语速较快、声调较大的语音可以给用户紧张急促的感觉,在说话时加入笑声能给用户一种轻松愉悦的感觉。

5.1.3 GUI和VUI融合时会遇到的问题

通过 5.1.2 节的介绍,相信读者对 GUI 和 VUI 的特点已有所了解。这两种交互方式单独使用不会有问题,一旦融合在一起就会有很多问题需要解决。

1. 两种交互方式的信息输出效率不一样

GUI 主要以文字、图像和视频为主,而 VUI 主要以语音为主。视觉接

收的信息量可以达到听觉接收的信息量的 100 倍以上。一旦 GUI 和 VUI 融合,就要考虑用户的注意力分配的问题。当用户的注意力集中在某一个通道时,其他通道获取信息的效率会迅速下降。例如,人在认真阅读和聆听同一个长文本时,视觉通道接收信息的效率会下降并和听觉通道同步;在认真阅读和聆听不同长文本时,阅读和聆听的效率会同时下降。这些都是因为人在阅读文字时,大脑其实在“朗读”并聆听这些文字,如果外界有相同的声音,大脑就懒得“朗读”文字而直接从听觉通道聆听文字,如果在“朗读”过程中一直有不同的声音干扰大脑聆听文字,大脑就会不知道如何处理不同通道接收的文字信息,有可能走神。但边看图像/视频边听声音的时候很少会出现这种走神的情况,除非声音和画面不同步。

2. 两种交互方式的操作对象不一样

GUI 的操作对象是获得焦点的界面元素, VUI 的操作对象是主语或宾语,两者不一定有关联。一旦 GUI 和 VUI 融合,就要考虑两者属性打通的问题。对于 GUI 控件来说,每个控件都是由对象实例化实现的,它们只有自己的基础属性,具体的业务逻辑由程序实现。例如 GUI 中的 Button (按钮),它只知道自己是一个 Button,有 Default、Focus、Hover、Active 和 Disable 状态,具体的业务逻辑和操作成功与否都是由程序中的回调函数决定的,Button 本身是不需要知道的。还有一个问题就是,Button 还可以由 Image、ImageButton 甚至其他控件替代,只要具备 onClick 属性,就可以是一个按钮,但这时的按钮有可能不知道自己是一个按钮。对于组件来说,控件升级成组件的自由度更高,有很多方式进行组合,这些组合方法不一定是常规组合。

但是在 VUI 中,每一句指令没有基础属性这个说法, VUI 才不管自己现在处于 Focus、Hover 还是 Default 状态,它只管这句指令是否被顺利执行,所以 GUI 和 VUI 关注的点是不一样的。如果要通过语音交互的方式操控界面里的元素,就需要根据元素的不同特点赋予它们更多的业务属性和状态属性,才能实现 GUI 和 VUI 的融合。

3. 表达方式不一样

GUI 能通过文本、图片、视频和布局结构表示不同的含义，而 VUI 只能通过基于文本的语音表达含义。GUI 和 VUI 在融合时，VUI 如何跟图片、GUI 的布局结构耦合在一起是关键。在耦合过程中难免会遇到一些歧义的地方，例如在 GUI 界面中，返回到上一个界面可以通过“返回”按钮或用意思相近的图标进行表达，在 VUI 中通过“返回”“上一页”指令也可以回到上一个界面，但是返回到“上一页”跟翻页中的“上一页”表述是一样的，这时使用基于文本的语音表达，系统并不能清晰地知道用户的真实意图，所以 GUI 和 VUI 在融合时需要把这些带有歧义的指令做好容错设计。

4. 交互流程不一样

GUI 是由界面元素和固定的信息架构决定的，它是结构化的。正因如此，如果由 GUI 提供个性化服务，例如订飞机票，GUI 需要为用户提供多种选择，这时界面会被大量的填空、下拉列表、按钮及层级关系填满，交互流程变得非常复杂。在 VUI 里，用户会根据平时自己的说话风格和想到什么说什么的习惯，说出高度个性化（非结构化）的语句，所以 VUI 的交互流程是随时可变的。在刚刚提到的个性化服务问题上，VUI 变得很简单，例如用户说“我要订一张明天 9 点飞往北京的机票”，只要意图设计得合理，VUI 可以直接给出符合用户期望的结果。GUI 和 VUI 在融合时都会对对方产生影响，例如 VUI 因为 GUI 拥有了返回逻辑，而 GUI 的界面逻辑需要考虑 VUI 可变化的交互流程来简化自己的界面。

5. 两种交互方式融合会带来互斥问题

GUI 的操作流程由界面元素和焦点决定，VUI 的操作流程由剧本决定，两者融合会带来两种互斥现象，分别是 GUI 和 VUI 操作互斥、GUI 和 VUI 反馈互斥。

GUI 和 VUI 操作互斥：在融合界面里，GUI 和 VUI 是强相关的。在

VUI 剧本中，GUI 操作可能会影响 VUI 剧本。例如 GUI 支持多任务切换，而 VUI 没有多任务并行这个概念，所以 GUI 执行切换任务时会导致 VUI 剧本错乱。

GUI 和 VUI 反馈互斥：VUI 在同一时间只能播放一条信息。在 VUI 播报时，如果 GUI 的反馈涉及 VUI 播报，则新的播报内容要么截断当前播报内容，要么等待当前播报内容结束后才播报，这两种方式都会对用户体验造成影响。

6. 两种交互方式融合会带来时序问题

当多个模态融合时，如何处理模态间的先后顺序是设计多模交互最核心的问题。站在用户的视角，用户认为自己说过的话、点击过的按钮都有先后顺序，这是很自然的事情，但是计算机并不这么认为，因为绝大部分的语音交互识别和理解都是在云端进行的，而 GUI 绝大部分的操作都是在客户端完成的，客户端和云端的数据传输产生的时差会直接影响多模交互的先后顺序。如果有一项操作由于时差的问题导致其响应和反馈晚于它的下一个操作，则整个交互流程很有可能出现重大问题，用户也会因此产生困惑。

以上这些问题都是笔者在实现 VUI 和 GUI 融合时遇到的问题，在部分问题上笔者已经找到较好的解决办法，在本章后续的内容中将会介绍。

5.2 GUI和VUI融合的三种实现方式

一般来说，多模交互中的 VGUI（VUI+GUI 的简称）有三种实现方式，分别是应用级语音交互、可见即可说和系统级语音交互。真正对多模交互有用的实现方式是系统级语音交互，下面介绍这三种实现方式的区分。

5.2.1 应用级语音交互

应用级语音交互的意思是，在语音交互的过程中，系统会调出一个语音应用遮盖当前界面，用户只能对语音应用进行操作或退出语音应用，因此语音应用和其他应用都是互斥的。以 iPhone 上的 Siri 为例，Siri 是一个信息中枢系统，它会把系统和第三方应用里所有的语音技能和需要的信息都集成在自己的应用里，用户使用 Siri 时必须暂停操作当前应用。应用级语音交互不需要知道当前发生了什么，这是它的第 1 个特点：应用级语音交互会脱离当前应用和场景。

应用级语音交互的前身是独立的语音交互系统，也被定义为语音智能助手，例如苹果的 Siri、亚马逊智能音箱 Echo 上的 Alexa 和 Google Assistant。这些公司打造语音智能助手的初衷是建立一个语音生态系统，不依赖 iOS 和 Android，也不依赖图形界面，能够独立工作，但需要第三方应用重新为这个语音交互操作系统设计一套语音应用和交互剧本。这是应用级语音交互的第 2 个特点：应用级语音交互由剧本决定交互路径。

后来，尽管出现了各种带屏智能音箱，例如亚马逊的 Echo Show、百度的小度在家和阿里巴巴的天猫精灵，但是它们搭载的语音智能助手仍然是之前那套语音操作系统。这些设备上的 GUI 和 PC/移动端的 GUI 不一样，PC/移动端的 GUI 是为了提升阅读效率而产生的，而带屏智能音箱的 GUI 是 VUI 的辅助和补充手段。

读者有没有发现图 5-1 和图 5-2 的带屏智能音箱界面中的字号都比较大？前面提到，在认真阅读和聆听同一个长文本时，文字阅读的效率会下降，视觉通道接收信息的效率会下降并和听觉通道同步，所以用带屏智能音箱边听边看一大堆内容是不现实的，而且用户有可能从远处看设备上的内容，因此设计师把界面中的字号放大，简化界面的排版布局。这也是应用级语音交互的第 3 个特点：应用级语音交互以听觉通道为主，弱化图像用户界面的信息。



图 5-1 Google Home Hub (图片来源: Google 搜索)



图 5-2 Echo Show (图片来源: 亚马逊官网)

当 Siri、Google Assistant 这些独立的语音操作系统依附在 iOS 和 Android 上时,它们自然而然地降级的为一个独立应用,所以称它们为应用级语音交互。由于自然语言理解和全双工技术尚未成熟,以及用户的双眼和双手就在手机屏幕前,所以在应用级语音交互中,语音交互不一定全程参与。以 Siri 和 Google Assistant 为例,用户通过语音启动任务后,随后的步骤允许用户通过触摸屏进行交互,图 5-3 展示了用户在 Google Allo 上发起了语音交互后,Google Assistant 为用户提供了基于 GUI 的多个选项。这是 VGUI 的第 1 个特点:多通道结合使用可以提升工作效率。

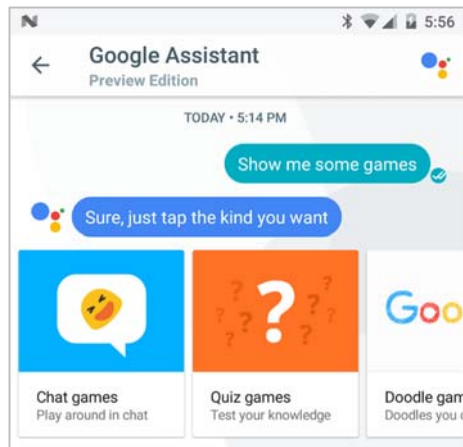


图 5-3 Google Allo

Siri 和 Google Assistant 可以通过对话流即对话用户界面（Conversation UI）的方式显示完整的上下文，图 5-4 展示了用户与 Google Assistant 对话的上下文内容。这是 VGUI 的第 2 个特点：显示对话内容有利于用户记忆和理解对话内容。

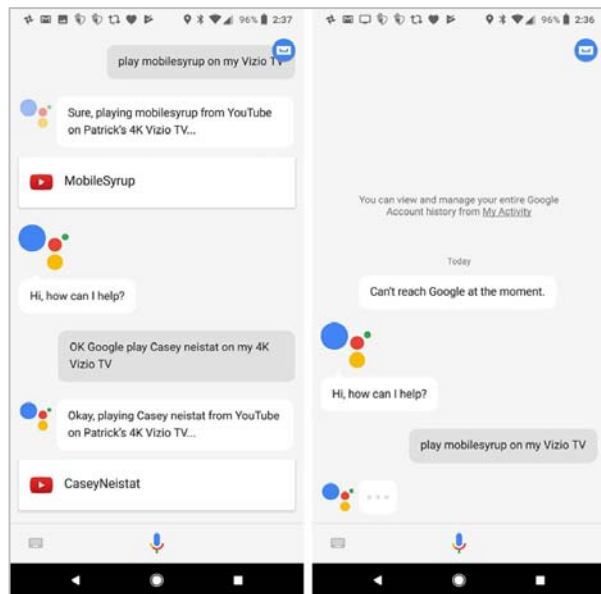


图 5-4 Google Assistant

以查询一周天气为例,不带屏的智能音箱会把未来7天的天气信息全部播报出来,播报时长可以高达1分钟;在带屏设备上,例如图5-5的Siri,未来一周天气可以只播报概要,每天的具体天气信息可以让用户自行阅读。这是VGUI的第3个特点:结合视觉通道输出信息可以减少语音播报唠叨。



图 5-5 iOS13 Siri 天气播报

从内容的角度来看,应用级语音交互只是通过GUI把VUI的内容可视化,同时降低了GUI的阅读效率。从交互流程的角度来看,图形界面交互和语音交互并不能很好地兼容,应用级语音交互只允许用户选择一种通道进行输入。尤其是微软的Cortana,当用户发起语音交互后,如果点击了Cortana应用内的任何按钮,就无法返回到语音交互流程上,因为此时Cortana的麦克风图标已被禁用。从技术实现的角度来看,模态之间的相互转换要求所有的输入和输出系统必须完全理解用户的所有状态和行为,这需要在系统底层提前定义好各种规则。基于图形界面的操作系统比语音交互系统成熟数十年,语音交互已经无法在拥有几千万行甚至上亿行代码的图形操作系统上深

度嵌入自己的代码且不会出错，无论是苹果公司还是 Google 公司都不敢冒这样的风险，更何况他们的图形操作系统团队和语音交互团队是完全独立的。因此，应用级语音交互算不上真正的 VGUI 融合。

5.2.2 可见即可说

可见即可说的意思是，在界面上看到什么元素，只要说出它的名字，系统就会通过模拟点击的方式操作该元素。例如，界面上有一个蓝牙开关，用户只要说“打开蓝牙开关”或“关闭蓝牙开关”，系统就会模拟点击蓝牙开关的热区，使开关发生变化。这是可见即可说的第 1 个特点：可见即可说用视觉通道接收信息，用语音的方式输出信息。第 2 个特点是：可见即可说可以在任意界面上使用。总结在一起就是 VGUI 的第 4 个特点：VUI 可以随时随地操控 GUI 上的界面元素。

可见即可说和应用级语音交互有着本质的区别。第一，应用级语音交互是一个独立的 VUI 操作系统，是汇集所有信息的中枢系统，而可见即可说属于 GUI 操作系统的辅助手段之一，它提供语音交互的能力来帮助用户解决偶尔无法双手操作图形界面的问题。第二，应用级语音交互不依赖连续对话的能力，而可见即可说若仅靠唤醒词和单轮对话，则体验较差，所以它和应用级语音交互有着不同的交互方式。这是可见即可说的第 2 个特点和 VGUI 的第 5 个特点：VGUI 依赖连续对话的能力。

可见即可说只需要语音识别和正则表达式匹配就能实现对界面元素的模拟操作，1992 年李开复博士在美国公开展示其开发的语音识别系统也运用了该原理。可见即可说不需要复杂的自然语言理解技术就能实现，正因如此，可见即可说不支持交互剧本、意图识别、业务逻辑理解等功能，一般用于简单的车载系统上。如果想在自己的手机设备上体验可见即可说的语音交互功能，在 Android 设备上可以下载 Google 的官方应用 Voice Access，然后在系统设置的无障碍设定中选中 Voice Access，就能用一些比较简单的语音

指令操控 App 中的控件。在 iPhone 上，可以在系统设置的辅助功能中打开语音控制的开关，但是截至本书出版前，iPhone 的语音控制能力有限，仅支持图 5-6 中的语音指令。因此，可见即可说的第 3 个特点：可见即可说不支持复杂的意图识别。

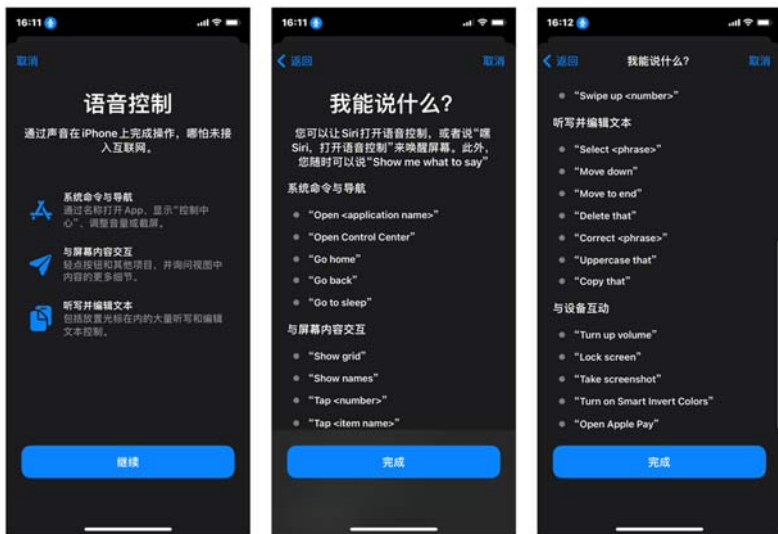


图 5-6 iPhone 的语音控制界面

总之，可见即可说只是通过 VUI 操作 GUI，其能力非常有限。正如 5.2.1 节提到 GUI 和 VUI 的交互流程不一样，可见即可说只能让用户看着订飞机票界面的交互组件一句一句地说出来，不能让用户一句话把一项任务做完，更不能让用户一次性完成多个任务，所以仍然算不上真正的 VGUI 融合。

5.2.3 系统级语音交互

系统级语音交互可以简单地理解为应用级语音交互和可见即可说的结合体，通过各自的优势弥补了对方的劣势。系统级语音交互的能力如下。

- 系统级语音交互属于 GUI 操作系统的一部分，拥有连续对话的能力，可以随时随地操控 GUI 上的界面元素。

- 鉴于第 1 点，系统级语音交互不会脱离应用和场景而单独使用，它的交互流程由剧本和界面元素决定。
- 系统级语音交互拥有意图识别和业务逻辑理解能力，因此系统可以理解用户的意图，也可以依据特定场景主动发起语音交互。
- 基于第 1、2、3 点，系统级语音交互具有信息汇集和理解的能力，是信息的中枢但服务于系统和各个应用，它把收集到的信息重新分发给各个应用。
- 基于第 4 点，系统级语音交互能突破界面的限制，可以随时随地跳转到任意应用和界面上。
- 系统级语音交互能显示用户和系统的对话内容，有利于用户记忆和理解对话内容，并且减少语音播报的唠叨。

系统级语音交互能兼顾 GUI 和 VUI 的优点，提升 VGUI 的工作效率，是真正的 VGUI 融合。最重要的是，它解决了所有数据和任务都集成在一个应用里的问题，能把数据和任务回归每一个应用，这才是操作系统该有的做法。以系统级语音交互为代表的多模融合需要重构系统框架且有强大的技术支撑才能被实现，而且它在手机和 PC 上没有太大的实质性作用，仅有少数公司会去探索系统级语音的交互实现方式，例如 Google 的 Duplex on the Web 技术和 iOS 14 中 Siri 的技术。多模融合真正发挥作用的领域是汽车、AR、VR 和跨设备交互，提前探索能沉淀更多关于系统设计和人机交互设计的知识，这也是笔者编写本章内容的初衷。

5.3 VGUI设计原则

VGUI 融合后重要的不是界面设计，而是用户的交互行为和目的，建立以用户为中心的设计原则能更好地理解这一点。除要考虑 5.1.3 节中提到的问题外，以系统级语音为核心的 VGUI 还有很多细节需要解决，下面 8 条设计原则可以帮助解决上述问题。

1. 交互是一种行为，它具有目的性

这句话是整个 VGUI 融合的核心。要知道用户每操作一个控件、每跳转一个页面，这些行为的背后都是有目的的。用户在意的是系统和应用能不能帮助他们达成目的，操控 GUI 的控件和页面只是辅助用户完成他们目的的手段之一。同理，VUI 的剧本也一样。因此，VGUI 融合后重要的不是界面设计，而是用户的交互行为和目的，它们会影响 GUI 控件/的文案设计，详细内容会在 5.4.1 节讲述。

2. 每种交互方式都能持续工作

该原则更多针对的是语音交互能力。在多模交互中，如果每一次语音交互都需要唤醒语音识别能力，就会影响整个操作流程和用户体验。所以，VGUI 融合的前提是系统/应用拥有全双工语音交互的能力，系统/应用能持续一段时间倾听用户所说的话。为了避免泄露隐私及噪声对系统的影响，用户一段时间内不说话，系统可以主动退出语音交互。

3. 每种交互方式统一以 GUI 为参照对象

在 VR、AR、车载系统中，无论是眼动操作、隔空手势操作、实体按钮、操控手柄还是 VUI，都应该以 GUI 为参照对象进行设计。这是因为：第一，视觉通道接收的信息占全部感官接收的信息的 83%，如果用以听觉为主的 VUI 为参照对象，则该交互框架能输入和输出的信息量会直接锐减；第二，GUI 显示的内容可以维持静止状态，而 VUI 无法做到这一点；第三，GUI 中有丰富、成熟的控件和组件，可以跟各种交互方式进行绑定。

以 GUI 为参照对象不代表各种交互模态只能和当前界面进行交互，它们也可以跟“不可见”的界面和功能进行交互，甚至绑定系统的全局功能。例如，旋转音量按钮可以随时提升/降低系统的音量；绑定了关闭功能的手势可以直接退出当前界面；通过语音交互可以随时切换到其他应用里的任意界面，或者执行某个界面里的某项功能（只要用户记得）。

以 GUI 为参照对象的 VUI 剧本也要考虑 GUI 的交互方式,例如返回和跳转逻辑,原先没有返回逻辑的 VUI 也能支持返回逻辑。相应的,当 GUI 中的任务或界面突然发生跳转时,VUI 剧本也应该跟随之变化。在设计 VGUI 信息架构时,设计师应该考虑 VUI 的特点,包括一句话完成一项或多项任务,以及说话顺序因人而异的特点。

切记,在 GUI 中不能打断交互任务和事件,其他交互方式同样不能被打断。在 GUI 中,有些用半透明黑色蒙层遮罩主界面的弹窗和对话框用于安全警示、重要内容确认等交互流程,用户只有在当前界面完成任务后才能返回主界面。在这种模式下,其他交互方式也需要遵循该交互组件的设计原则。因此,可以随时跳转到其他页面的 VUI 应该收敛意图识别的范围,只允许用户聚焦于当前任务上。

4. 每种交互方式相互配合,取长补短

每种交互方式相互配合的前提是互不干扰。前面内容中提到了应用级语音交互和其他应用是互斥的,它会调用一个模态窗口遮盖当前界面并显示语音应用,用户只能对语音应用进行操作或退出语音应用。在 VGUI 融合时,VUI 不应该遮住 GUI 的重要区域,同时可以在 GUI 任意界面中被唤醒。因此,VUI 的展示只能以浮层的形式展示,只占用 GUI 边缘的一小部分区域,例如屏幕顶部的状态栏或底部区域,而且展示 VUI 的同时用户能对主界面进行操作。

VUI 的第 1 个缺点是交互状态不明显,所以 VUI 的交互流程中建议配合 GUI 或其他 LED 灯光效果强化 VUI 的状态显示。VUI 的第 2 个缺点是受限于工作记忆,语音播报的内容、句式和语法结构必须简单,笔者建议内容播报尽可能控制在 10 秒以内(中文和数字约为 40 字以内),包含的信息尽量控制在 3 项以内。如果 GUI 内容支持语音播报,那么播报的内容应该和显示的文本保持一致;建议在 GUI 上显示当前播放进度,这样能避免用户四处查看现在读到哪里。

那么，GUI上的所有信息是否相应地只显示3项内容呢？例如，菜单上只提供3个选项、工具栏上只显示3个图标。显然不是的，因为这些内容并不是短暂地显示在屏幕上，它们可以被眼睛来回地扫视和重新阅读，所以用户在交互过程中没必要记住全部信息。正确的做法是，对于选项过多的菜单或列表，VUI可以优先播报对于用户来说重要的前3项，然后询问用户是否继续播报，与此同时，用户还可以通过触控的方式从GUI获取信息。

GUI的缺点很明显，它需要用户看着屏幕才能正常交互。在驾驶过程中，司机开车时低头很容易发生事故，因为在2秒的时间内一辆时速100千米的汽车能开出54米的距离。根据美国弗吉尼亚理工大学交通学院在2009年的调查结果显示：如果卡车司机在开车时发短信，则发生车祸的概率是正常驾驶时发生车祸的概率的23倍。所以，美国国家公路交通安全管理局(National Highway Traffic Safety Administration, NHTSA)发布的*Visual-Manual NHTSA Driver Distraction Guidelines*中提到，车载系统的GUI设计尽量能让司机在行驶过程中快速完成任务，在一次任务过程中眼睛单次离开道路不应该超过2秒，总时间不应该超过12秒。在上述规定下，传统车载系统的界面和功能都非常简单，但是引入车联网和辅助驾驶相关技术后，新型车载系统的功能逐渐丰富起来，界面也逐渐变得复杂，NHTSA规定的单次2秒、总时间12秒的相关规定很难被执行。因此，在汽车场景下，让用户不看界面也能和系统交互的VUI变得越来越重要。在2020年10月小鹏汽车发布的“2020年智能数据报告”中显示，小鹏汽车的语音交互每天人均有效唤醒次数为7.1次，月均使用率为99.74%¹，这个数据能充分证明VUI在汽车领域有多么重要。为了弥补GUI的缺点及提升驾驶场景中用户和系统的交互安全，GUI和VUI可以这样融合：

- VUI可以操控GUI的界面和功能，尤其文本输入功能；
- GUI显示的文本内容允许VUI播报相关内容；

¹ 数据来自小鹏汽车2020年官方发布的《智能数据报告》。

- 由 VUI 播报完整信息，GUI 通过排版显示重点信息；
- 基于 VUI 的声纹识别能直接省略用户在 GUI 中输入密码的交互步骤。

第 1、2、4 点都很好理解，这里笔者重点对第 3 点进行解释。第 3 点利用了听觉通道和视觉通道各自接收信息的总量和时间差。举个例子，在驾驶过程中司机最先从听觉通道获取信息，如果司机认为这项信息不重要，就可以通过方向盘上的实体按键取消播报；如果司机认为重要，就可以瞥几眼屏幕上的内容。这时，将重点信息结构化排版的 GUI 不仅能避免 VUI 的播报会干扰用户的无声阅读，还能让用户在最短时间内知道完整信息是什么，提升整个接收信息的效率。

当用户没有注视操作对象发起语音交互时，需要用户说出操作对象的完整名称及语音播报完整的反馈是没有问题的，但是如果用户盯着操作对象，是否还需要用户说出操作对象的完整名称和语音播报完整的操作反馈？很明显不是的。因为用户注视着操作对象就知道反馈结果是什么，这时候若干秒的语音播报反而成为“鸡肋”。连续的语音反馈对于用户来说更是一种噪声，而且降低用户的操作效率。所以，笔者推荐用简短的音效替代 VUI 操作 GUI 时的语音反馈，这样不仅能减少对用户的干扰，而且能让用户形成听觉记忆，用户即使不看屏幕听音效也能知道发生了什么。

5. 以用户当前操作对象为目标发起交互流程

在多模交互中，最重要也是最麻烦的操作是操作对象的切换，因为有可能意味着任务和上下文发生变化。在语音交互中，主语或宾语是操作对象；在触屏 GUI 中，手指触碰的地方是操作对象；在 VR 和 AR 界面里，视线追踪、隔空手势、操控手柄显示的焦点是操作对象。在设计过程中，我们应该如何考虑操作对象切换的问题呢？

操作对象不只是控件和组件，它还可以是一个页面，甚至是一个任务流程。VGUI 融合的关键是将 VUI 意图中的主语或宾语和 GUI 里的控件、组

件或容器进行绑定，系统通过操作对象的对照就能知道 GUI 和 VUI 是否在操作同一个操作对象。以 GUI 为参照对象的好处是能让操作对象显性化，用户通过焦点的切换就知道自己的交互操作是否被系统正确识别，在多任务、多窗口里也能知道现在哪个任务或窗口被激活。

当用户通过不同的交互通道处理操作对象时，我们应该考虑以下细节：

- 当不同通道的操作目标为同一个操作对象时，全部通道的交互流程应以最新的操作为准，同步更新各自交互流程和相应的数据；
- 当不同通道的操作目标不是同一个操作对象时，各个操作对象保持原有的交互流程和相应数据；
- 当操作对象与其他对象产生联动时，需要同步更新联动对象的全部通道的交互流程和相应数据；
- 当操作目标为多个操作对象时，需要同步更新相关对象的全部通道的交互流程和相应数据；
- 如果新的操作对象有后续的交互流程，则应该在 VUI 或 GUI 中体现出来。

6. 明确告诉用户当前的交互流程到哪里了

每种交互方式都具备“选中目标”“执行过程”“结果反馈”3种属性。相信读者对 GUI 中各种控件的按下态、加载态都很熟悉，而在无法感知的交互过程中，例如 VUI 执行过程中的聆听、识别和加载状态，以及使用没有数字刻度的旋转按钮，这些细节很容易被设计师忽略。

“选中目标”能让用户和系统清晰地知道当前操作的对象是谁；“执行过程”能让用户知道当前的交互进度到哪里，避免用户产生焦虑；而“结果反馈”应该考虑“成功执行”和“无法执行”两种情况。“成功执行”很好理解，而“无法执行”比较特别。GUI 的好处是每个控件的状态都能被用户看到，用户每一次交互都是围绕控件、组件或容器进行的。它们能引导和限制

用户的交互流程，例如一个滑动条，用户滑到最左侧就不能而且不允许继续滑动了。但是 VUI 做不到，因为用户看不到滑动条的上下限在哪里，而且发出的指令不一定在系统支持的指令集中。所以，“无法执行”应该包括“业务支持失败”和“听不懂（没法分类）”两种情况。如果缺少“业务支持失败”这个细节，VUI 只懂得跟用户说“不好意思我听不懂”或“不好意思我无法执行”，就会引起用户的反感。

7. GUI 控件/组件应支持多种交互方式，如有差异建议增加说明

在多模交互下，不同类型的操作控件/组件应有不同的 VUI 意图和流程来支持，文本类型的控件支持语音播报功能，详细的 GUI 控件/组件设计会在后文详细讲述。在自然的多模交互中，用户在不同场景下有可能通过不同的交互通道完成相同的任务，所以在设计多模交互体验时应该做到完整的冗余设计。有些交互操作确实不太好实现冗余的设计，尤其是一些基于纯图标的按钮，或者全是复杂文字的链接，这时我们应该想办法在图标上增加文字，以及在链接前面增加数字，如果因为各种因素无法修改设计方案，笔者建议通过其他方式告知用户暂时无法支持其他通道的交互操作，这样能有效避免用户觉得这是一个 Bug。

8. 由交互管理器统一管理多种交互方式之间的操作和状态，包括容错管理、意图/界面切换

由于多通道之间的信息输入和输出存在不同效率、同步/异步及兼容/互斥的差异，因此构建交互管理器有助于管理多模态交互之间的状态，同时有利于管理第 5 点。简单理解就是，我们需要通过一个交互管理器来管理所有操作对象及交互通道之间的关系，它的主要作用是监控不同操作对象及交互通道产生操作数据时的先后顺序，然后将这些信息操作同步给所有交互通道，从而实现交互通道的管理。该作用和第 1.2.3 节提到的多模交互框架有异曲同工之处，但它没有做到不同交互通道之间的完全解耦，只是由一个交互管理器来管理不同交互通道之间的状态，能有效降低 VGUI 实现的难度和成本。

5.4 重构GUI操作

5.4.1 GUI控件文案设计

第 5.3 节提到“VGUI 融合后重要的不是界面设计，而是用户的交互行为和目的”。怎么理解行为和目的？用户完成一件事是目的，中间的步骤是行为。以注册账号为例，用户填写邮箱密码和单击“注册”按钮都是行为，最后一步单击“注册”按钮注册成功才是用户的目的。所以，用户每操作一个控件、跳转一个页面都是行为，最后一步操作是行为也是目的。VUI 指令既是用户的行为也是用户的目的，但是 GUI 不一样，因为控件、组件和页面属于计算机特有的产物，所以 VUI 和 GUI 融合的关键是围绕用户的行为和目的展开设计，并将此赋予 GUI。GUI 通过文案、图形和结构表示信息，一般来说，GUI 的文案和图形设计没有任何限制，下面以图 5-7 为例进行说明。



图 5-7 不同按钮的文案和形状

图 5-7 中的 3 个按钮代表 3 个操作，第 1 个是代表“红色 D62341”的选项按钮，第 2 个是代表“窗户已打开/关闭”状态的开关按钮，第 3 个是“在 Wi-Fi 连接时同步”数据的开关按钮。这 3 个按钮原则上没有任何问题，用户的点击成本和犯错成本都很低，但是如果结合 VUI 指令，那么选择第 1 个按钮说“红色 D62341”还是说“选择红色 D62341”？选择第 2 个按钮说“窗户已打开”“打开窗户已打开”还是“打开窗户已打开的按钮”？选择第 3 个按钮说“在 Wi-Fi 连接时同步”还是其他说法？

相信读者已经发现以上 3 个按钮的名称读起来并不顺畅。第 1 个按钮的问题是中英文和数字混合，读起来很拗口，而且朗读时间较长。如果按钮名字中混有复杂的英语单词，例如 *prioritize*，用户有可能不能把按钮上的单词

朗读出来。如果将第 1 个按钮的文案改成专有名词或数字，将 VUI 指令改为“选择法式红”或“选择第 1 个”，如图 5-8 所示，读起来是不是容易很多而且节省时间呢？

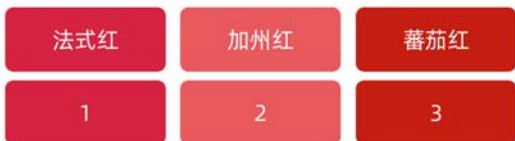


图 5-8 第一个案例修改后的文案

第 2 个开关按钮的问题更严重。无论是“窗户已打开”“打开窗户已打开”，还是“打开窗户已打开的按钮”，这句 VUI 指令都是有问题的。“窗户已打开”这句话表达窗户的状态，它不是一种行为，所以构成不了 VUI 指令，如果强行构成 VUI 指令，用户读起来会很困扰；“打开窗户已打开”这句话语法有问题；“打开窗户已打开的按钮”句式很长，这么长的指令效率很低，尽管“打开 XXX 的按钮”语法没有问题，但是用户第一直觉是“所见即所说”，所以用户第一反应不会想到宾语是这个按钮。因此，这个按钮的文案应该通过“标签+开关”组件化的方式进行改进。

第 3 个按钮的问题和第 2 个按钮的问题是类似的。“在 Wi-Fi 连接时同步”这句话包含了较复杂的语法结构，而且字数过多。其实这句话想表达的意思是在打开/关闭 Wi-Fi 时同步数据，这个按钮的文案应该通过“标签+开关”组件化的方式进行改进，如图 5-9 所示。



图 5-9 第 2 个和第 3 个案例修改后的文案

从以上 3 个案例我们可以看出，在 VGUI 中，操作型控件的文案设计会受到限制，主要限制有以下几点。

1. 文案符合“可见即可说”的原则

“可见即可说”包含两个设计理念，第一个是用户看得懂，第二个是用户朗读顺畅。对于用户来说，朗读文字的前提是看得懂文字，例如英语单词“serendipity（机缘巧合）”，不懂英语的用户是无法朗读出来的，所以建议设计的文案以用户熟悉的语言为基础。另外，以图 5-7 所示的控件文案为反面教材，用户读起来比较费劲，因此在设计 VGUI 文案时需要考虑用户读起来是否拗口。

2. 文案中尽量不要出现汉字、数字、标点和大小写字母的混合

对于语音识别系统来说，识别多语言和数字混合是一个很大的挑战，例如文案中存在“1”“e”“衣”，不仅 ASR 识别的准确率会很低，而且用户读起来会非常拗口，所以不建议在文案中混合汉字和数字。标点和大小写字母是辅助文字记录语言的符号，是书面语的组成部分，但在说话时并不会出现这两种元素。以图 5-10 左侧 Wi-Fi 列表为例，图里任意一个 Wi-Fi 名字读起来都非常拗口。以“Xiaomi_2468”为例，用户有可能将其读为“大写 Xiaomi 下画线 2468”，这时是有歧义的，究竟是只有“X”是大写，还是“xiaomi”都是大写，而且系统有可能把 ASR 中提及的符号“大写”和“下画线”理解为这句话中含有中文“大写”和“下画线”，最终结果就是理解失败。所以，不建议在操作型控件中出现汉字、数字、标点符号和大小写字母的混合，但不可避免的是 Wi-Fi、蓝牙及地图列表中一定会出现多种符号的混合，因此建议在列表前增加数字作为标识，以图 5-10 右侧列表为例，用户只需要读出“连接第 2 个”就可以了。最后，也不建议在多符号混合的密码输入框中增加语音输入的功能。



图 5-10 修改名字前和修改名字后的 Wi-Fi 列表

3. 文案之间避免过于相似，文案长度尽量控制为 2-6 个字符

文案越长，文案相似的可能性就越小，语音指令的命中率就会越高。例如，GUI 中有“开关”和“开启”两个文案，两者的区别只有一个字，如果考虑噪声和系统文字纠错的影响，有可能用户说了“开关”但系统误判为“开启”，给出不一样的语音指令，导致系统反馈不符合用户预期。在 VGUI 中，GUI 的控件和组件都能和相关 VUI 指令进行绑定。从用户调研的数据中发现，当用户朗读文案时，文案越长越浪费用户的注意力，部分用户读到后面时速率会有所下降，因为他们觉得这种行为比较呆板。从以往的设计经验来看，笔者建议每个控件的文案尽量控制为 2~6 个字符，既能涵盖绝大部分功能需要表达的信息，又能保证语音指令命中的准确率，大部分用户朗读时间在 1.5 秒以内，保证了 VUI 指令的高效性。

最后补充一下，从计算机的角度来看，两个字的文案识别准确率较低，但考虑到很多词语都是由两个字组成的，例如“登录”和“注册”，所以我们依然要把两个字的文案纳入 VGUI 文案设计规范中，这时需要语音系统对这些词语的识别做针对性的优化。

4. 文案结构简单

前文提到 VUI 和 GUI 融合的关键是围绕用户的行为和目的展开设计，从 VUI 指令的角度来看，指令要符合“动宾短语”和“把 XX 设置为 YY”

两种结构，因此 GUI 文案可以围绕这两种结构进行设计。例如，“打开空调”“关闭蓝牙”符合动宾短语结构；而“把温度降低 15 度”“降低温度 15 度”“温度降低 15 度”都符合“把 XX 设置为 YY”。这里笔者做一个补充，中文语法本身就比较复杂，加上各个地方的方言语法不一样，导致中文语法分析难上加难，在这里笔者吃了比较大的亏，所以建议读者不用太纠结中文语法问题。

以“打开/关闭蓝牙”为例，在设计 GUI 时有三种常用方法：第一种是在按钮上显示“关闭蓝牙”，点击按钮后文案变为“打开蓝牙”；第二种是通过带有“蓝牙”文案的标签和开关两种控件来表示；最后一种是在按钮上显示“蓝牙”，然后通过按钮点击状态/不可点击状态来表示蓝牙是否被开启。笔者在做第三种方法的用户测试时发现，当用户看到按钮想用语音操作它时，首先想到的是读出按钮上的文案，尽管按钮上没有“打开/关闭”两个文案，但用户说出指令时通常会将其补全；当按钮上的文案超过 4 个字时，部分用户很有可能不会补全“打开/关闭”等动词，因为他们觉得字数太长，朗读起来比较麻烦，而且他们认为读出来的指令已经足够清晰，系统能理解他们说的意思是什么。该测试结果说明，只要操作简单明了，设计师不一定要将动词显示在 GUI 文案上。

在 GUI 操作系统中，图标按钮是最常用也是设计师最喜欢用的控件之一，尤其是在国际化产品设计中，图标按钮能解决一部分多语言表达的问题。但是在 VGUI 融合过程中，图标按钮如何与 VUI 指令绑定在一起成为设计的最大难题，因为绝大部分的图标代表一个名词，而 VUI 指令更多的是“动词+名词”的结构。用户看得懂图标不代表能立刻说出图标的意义；如果看不懂，用户还能点击图标尝试理解，但在语音交互中他们完全不知道如何把图标表述出来。笔者从设计师常用的 iconfont 网站上下载了几个图标，读者认为图 5-11 所示的这些图标代表什么意思？



图 5-11 笔者下载的图标

这些图标均能代表多个意思：第一个图标可以表示“白天模式”和“亮度调节”；第二个图标可以表示“调节”和“设置”；第三个图标可以表示“菜单”和“设置”。当用户第一眼看到这些图标时，说出的指令很有可能和设计师设计时想表达的含义不一致。因此笔者建议，在 VGUI 中能不用图标表示的功能就尽量不要用图标表示，如果真的要使用，请提前做好用户测试，并且在图标底部增加文字信息辅助用户理解。

5.4.2 GUI组件和容器文案设计

如果只通过单个控件无法表达完整的信息，那么设计师应该考虑通过组件进行表达，例如单独的滑块（Slider）只能代表操作的上下限和当前状态，需要和标签或图标结合在一起才能让用户知道这个滑块的操作对象是什么，如图 5-12 所示。在设计组件时，组件内部结构设计要符合格式塔原理（Gestalt Theory），关键内容尽量保持在 12 个字以内（朗读 12 个字 3 秒左右），这样有助于用户高效说出相关指令。



图 5-12 调节温度滑块

一般来说，容器用来承载不同的控件和组件，本身意义不大。在设计师眼里，弹窗、模态框、对话框、窗口等专业术语有着不同的含义，因此设计师会将它们做区分并用于不同的设计中。但是在普通人眼里，它们可能是一样的，都是“弹窗”，所以设计师在设计 VGUI 时要格外注意，应该站在用

户的角度考虑用户需要什么，而不是纠结于各种设计规范、专业术语和控件类型。

列表（List）和选项（Item）的设计在容器中尤为重要。以图 5-10 所示的 Wi-Fi 列表为例，用户朗读一个 Wi-Fi 名字是一件很费精力的事情，建议在列表前增加数字作为标识。建议每个选项只有一个操作，这样有助于用户知道怎样开口。详细的组件和容器设计会在 5.4.3 节进行介绍。

最后谈谈页面。由于有些指令可以直接跳转到任意一个页面，因此页面的命名方式很重要。举个例子，当用户说“打开微信朋友圈”时，完全没有问题；但是用户说“打开微信我”，就有问题了，首先这句话不知道是什么意思，其次用户不会第一时间想到这种说法，所以不要给页面命名一些让用户说出来感到疑惑的名字。另外，有些命名方式应该支持多元化的表达。例如“钱包”在粤语里被叫作“银包”，粤语里几乎不会用“钱包”二字，因为说起来非常别扭，所以无论用户说“打开微信钱包”还是“打开微信银包”，系统都应该听懂这是要跳转到微信钱包的界面。因此，意思相同的词语做到相互映射更符合用户的预期。

5.4.3 特殊的控件和组件

有一部分控件和组件是专门为 GUI 设计的，在 VGUI 中并不适用，需要重新构造它们的结构。以 Android 控件为例，图 5-13 所示的菜单（Menu）、微调框（Spinner）等控件有较多选项，但除第一个选项外，其他选项对于用户来说是不可见的，而且这些控件相对独立，所以用户第一时间不知道怎样通过语音交互的方式选择它们，笔者建议在应用里少用这种类型的控件，如果确实需要使用，建议绑定一个固定标签来提示用户该怎样说。另外，默认显示 3 秒后自动消失的消息框（Toast）控件，在 VGUI 里适合使用声音进行反馈。



图 5-13 菜单和微调框

文本框（Text fields）是用户输入内容时最重要的控件，让用户通过触控、语音两种方式在 VGUI 中输入内容是整个体验的关键。在 GUI 中，语音输入一般由输入法提供，在 VGUI 中，推荐将语音输入的功能融入到输入法应用中，这样方便用户通过输入法的功能编辑文字。需要注意一点，在语音输入时是不支持空格、换行、选中、删除和退出等文本编辑操作的，原因是在语音识别过程中上述词组会被识别为输入内容之一，除非系统提前将这些词语标识出来并转换成相应的操作，但这样会导致用户永远输入不了以上内容，笔者不建议这样设计。在用语音输入文本的过程中，可以结合时间停顿、实体按钮及图形界面按钮来使语音输入退出。搜索框是文本框的一种延伸，除具备该有的功能外，还可以通过指令“搜索 XXX”让用户直接发起搜索。

选择器（Pickers）一般包含两个以上选项，例如时间选择器和日期选择器（如图 5-14 所示），如果按照“可见即可说”的方式，用户是很难对此进行操作的，因为在 GUI 中，选择器只需要将数值滚动到指定位置即为确定选项，但在语音交互中可能需要说“6 月，确定，6 日，确定”（用户可能还不知道要说“确定”这个指令来切换到下一个选项），这样的说法是很不友好的，所以建议通过意图分析识别用户的整句话，然后通过技术手段将每个数值解析出来并同步给 GUI。同理，操作系统中的电话拨号键盘也应该如此处理。

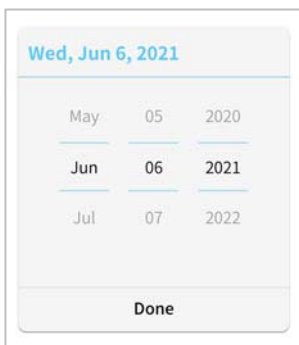


图 5-14 日期选择器

5.5 VGUI核心设计要点

5.5.1 全局指令和局部指令的权衡

前面提到用户完成一件事是目的，中间的步骤都是行为。行为是局部指令，目的是全局指令。在 GUI 中，任何操作事件都可以分为行为和行为/目的，该如何理解呢？这里先卖个关子。

首先我们需要了解应用和界面是怎样呈现给用户的。以 Android 为例，每一个应用都有自己的生命周期，当应用在前台界面时，我们可以认为该进程（Progress）处于激活状态，在后台时就不一定了，因为操作系统中有一个名叫 LMK（Low Memory Killer）的进程“杀手”，当系统处于低内存状态时，LMK 会释放那些不太重要进程的内存，让系统运行更加流畅。当进程被“取消”或没被启动时，我们可以认为当前应用没有被激活，这样设计的目的是避免内存被某个应用滥用，导致系统卡顿，甚至崩溃。为了避免占用过多的内存，有些应用只有当前界面显示的内容才会被存储到内存中，很有可能当前界面的下一屏内容都没被加载至内存，那么我们就不用期待下一个流程的界面甚至上一个流程的界面会被激活了。

从进程的角度来看，语音交互指令需要分为全局指令和局部指令。全局

指令的意思是它能在系统内全范围执行，不会被内存管理取消。假设一个应用的所有语音指令都有一个控件或组件绑定，全部指令都设计为全局指令，这等于默认该应用的所有界面和元素都要在内存中等待。如果所有的应用及系统功能都这样，系统及内存就会“吃不消”，所以并不是所有的控件和任务都能成为全局指令的。于是就有了局部指令，局部指令只对当前仍在内存里的界面元素和交互任务负责。另外，我们还应该从设计的角度来看，例如分享文章、删除选项等与当前内容有关的功能，上一页、下一页、切换导航标签等和当前界面有关的操作，这些功能和操作都和上下文有关，当它们脱离上下文后直接变成没有指定对象的操作，而且用户在看不到它们的情况下大概率也不会直接喊它们的指令，所以它们不应该成为全局指令。

那么，哪些指令可以成为全局指令呢？我们可以从 MVC 框架的角度来考虑这个问题。MVC 即 Model View Controller，是模型 - 视图 - 控制器的缩写，它是一种软件设计中常用的设计模式，在维基百科中它们的定义如下。

模型：模型用于封装与应用程序的业务逻辑相关的数据及对数据的处理方法。模型有对数据直接访问的权力，例如对数据库的访问。模型不依赖视图和控制器，也就是说，模型不关心它会被如何显示或被如何操作。模型中数据的变化一般会通过一种刷新机制被公布。

视图：视图能够实现数据有目的的显示。在视图中一般没有程序上的逻辑，为了实现视图上的刷新功能，视图需要访问它监视的数据模型。

控制器：控制器起到不同层面间的组织作用，用于控制应用程序的流程，处理事件并做出响应。事件包括用户的行为和数据模型上的改变。

从定义可以看出，全局指令能对系统及应用全局响应，同时不依赖当前界面，跟模型的定义很像，所以在设计全局指令时，该指令最好能对数据产生直接作用。另外，部分指令可以跳转到任意界面，所以全局指令也和控制器有关。对于局部指令来说，它能对当前界面的所有元素及任务产生作用，所以局部指令也和模型、视图及控制器有关，但是它更偏向视图，因为视图

一定要存在于内存里。从不同角度来看，全局指令能直接对数据进行交互，局部指令通过视图对数据进行监视和修改，这也回到了本节一开始提到的“在 GUI 中，任何操作事件都可以分为行为和行为/目的”。任何交互都是行为，但只有能对“模型”里的数据产生作用的才是用户的目的。因此，局部指令可以理解为基于视图的行为，而全局指令可以理解为基于控制器、模型的行为和目的。

MVC 框架只是一种设计模式，它能帮助程序员更好地将代码进行抽象和分离，从本质上说，模型、视图和控制器都是你中有我、我中有你的，无法做到完美地分离。同样的，全局指令和局部指令也是类似的关系，但是将它们抽象并区分开来可以帮助设计师更好地判断哪些任务和控件对用户来说才是最重要的、哪些任务和控件可以对数据产生影响、用户是否期待这些任务和控件脱离该界面后还能运行。如果一个任务需要多步操作才能完成，那么我们是否应该为它设计一个便捷的交互流程？如果是，那么这个便捷的交互流程在不同界面都有可能管用。如果不是，那么该操作或该控件对用户来说只是完成任务中的其中一步，它不必随时做出响应。这样的思考方式对后续 VUI 指令和 GUI 控件、组件的绑定及响应区域会产生直接影响。

VUI 指令和 GUI 控件、组件的绑定会在第 5.5.2 节介绍，这里先介绍响应区域。前文提到在 VGUI 融合时，VUI 不应该遮住 GUI 的重要区域，同时它可以在 GUI 的任意界面中被唤醒，因此 VUI 只能以浮层的形式展示。如果 VUI 浮层要展示内容，该内容优先显示系统和用户之间的对话，即 ASR 和 TTS，但是有些对话内容可能牵扯到图文信息的展示或存在多个步骤的任务，那么这些内容一定和全局指令强相关，在这里笔者将该浮层定义为“VUI 全局展示区域”，它的放置区域如图 5-15 所示。

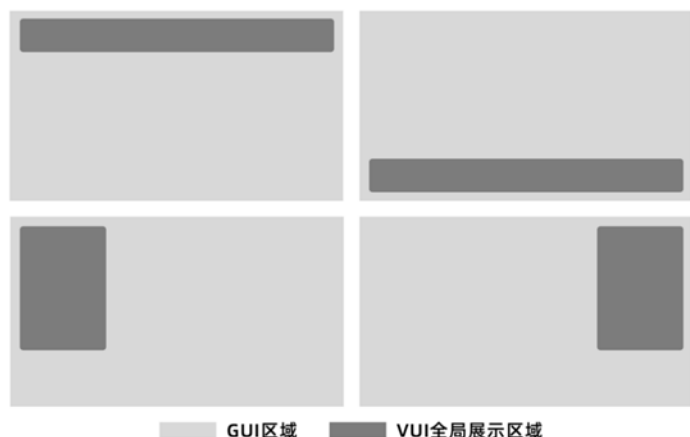


图 5-15 VUI 全局展示区域

VUI 全局展示区域的优势是能在任何界面响应。那么它和当前界面的关系是什么？在当前界面（任务）发起语音交互只有两种可能性：第一种可能性是用语音交互的方式替代当前的触控交互，该替代方法不会对任务及界面产生影响；第二种可能性是在当前任务下想通过语音交互的方式发起另外一项任务，这时系统存在两个任务，它们的关系要么是共存的（两个任务同时显示在界面里），要么是互斥的（旧任务界面切换到新任务界面）。

当然，以上两种关系由界面的空间大小及任务的优先级决定。对于不缺少屏幕空间的大屏交互来说，通过全局指令唤醒另外一个任务可以是完整的应用界面，后续所有交互流程可以在当前应用界面完成，这就回归到刚刚提及的第一种可能性，这是合理的设计。对于小屏交互来说，同时显示两个完整的应用界面已经是极限，这时共存和互斥变得非常重要。对于用户来说，追求高效和体验是很重要的，如果只是一个拥有若干步骤的任务，甚至是一句话即可搞定的任务就替用户切换整个界面，则直接降低了整个交互效率。因为应用没有用完就退出的说法，所以当前任务完成后，应用会一直停留在当前屏幕上，系统也无法帮助用户自动切换回旧的任务。同理，两个完整应用都显示在屏幕上也会出现类似的问题。VUI 全局展示区域没有这个烦恼，因为 VUI 拥有自己的生命周期，只要任务完成或用户长时间不交互就会自

动退出，同时在上文规定了该浮层不能遮住 GUI 的重要区域，所以它能较好地和当前任务共存。

在第 5.2.3 节中提到，系统级语音交互可以简单地理解为应用级语音交互和可见即可说的结合体，它具有信息汇集和理解的能力，是信息的中枢但服务于系统和各个应用，它把收集到的信息重新分发给各个应用。这也是将“共存”“互斥”绑定在一起。VUI 全局展示区域即不占据全屏的应用级语音交互，它的数据来源于各个应用。为什么应用愿意把数据给 VUI 全局展示区域呢？因为每个应用都希望自己的部分功能能被全局调用，所以 VUI 全局展示区域展示的内容一定是全局指令的内容。同时，有些全局指令的数据来源于云端，例如闲聊、百科等，这些在客户端没有相应的应用作为载体，VUI 全局展示区域为这些全局指令提供了展示载体。

如果不做信息中枢，那么系统级语音交互应该把收集到的信息重新分发给各个应用，这一点体现了“互斥”。这么做的理由也很简单，毕竟每个应用都不会把自己所有的数据和交互任务拱手让给系统来做决策；同时有一些交互任务步骤较长，或者任务需要较多的区域，甚至需要特定的控件才能做出响应，例如地图的展示，在面积较小的浮层内展示并不合适，所以界面必须切换到新任务上，这时当前界面语音指令主要以局部指令为主。

全局指令和局部指令的声音反馈是不一样的。对于全局指令来说，该指令有可能是在任何场景下发出的，包括用户不看着界面发出指令的场景，这时完整的语音反馈对于用户来说非常重要。但是对于局部指令来说，用户肯定是看着当前界面发出指令的，如果每一步交互步骤都有若干秒的语音播报，时间一长语音反馈无疑是一种噪声，而且会降低用户的操作效率。所以笔者推荐用简短的音效替代局部指令的语音反馈，可以有效降低对用户的干扰。其实，以上交互策略是为了避免用户在盯着屏幕看的同时还要听语音播报内容，重点在于用户是否在看屏幕，后续如果能通过视线追踪识别用户是否在看屏幕，语音反馈就可以做出动态调整，整个用户体验会变得更好。

综上所述，全局指令和局部指令是互补的，也可以说是互斥的。两种做法都能满足用户的需求，但哪种更好需要人为来判断，如果设计得合理，它们就是互补的关系；如果设计得不合理，两种做法都会相互“争功”，这时是互斥的。如何设计得合理会在后文中讲解。界面是否能跳转需要考虑很多细节，以下是笔者做的部分梳理。

（1）当上一轮对话属于单轮对话或多轮对话的结束部分时

- 当前意图和当前界面有关，不跳出当前界面。
- 当前意图和当前界面无关，跳出当前界面。
- 该意图强制跳转。

（2）当上一轮对话处于多轮对话过程中，而且该对话剧本拥有防跳出机制（不允许跳到其他对话剧本或界面）

- 当意图不支持跳出时，对话的上下文停留在当前对话剧本和界面中。
- 支持跳出的意图不支持跳回上一轮对话和相关界面。

（3）当纯语音播报相关的意图不影响当前界面时

- 如果新的操作对象有后续交互流程，则界面和任务应该跳到对应的流程上。
- 如果操作对象后续没有流程，则可以停留在当前界面。

5.5.2 控件/组件属性的重构

在 VGUI 中，VUI 和 GUI 融合后的控件或组件包含的属性如下：

```
{
  名字：控件或组件的 GUI 文案
  执行范围：只允许当前界面执行/允许其他界面执行
  控件状态：该控件或组件拥有的所有状态
  当前状态：
  区域内多模交互响应容错：当控件/组件互斥时，交互事件以客户端最后一次操作的时间为准
  允许交互：
    1.跳转事件
```

```

2.当前界面执行
    GUI 响应事件：显示远程响应成功动画效果
    成功执行 VUI 响应事件：绑定语音播报内容或提示音
    执行失败 VUI 响应事件：绑定语音播报内容或提示音
不可交互：
    GUI 响应事件：显示远程响应失败动画效果
}

```

{}：在 5.1.3 节中提到，按钮可以由 Image、ImageButton 及其他控件替代，只要具备 onClick 属性，就可以是一个按钮。对于组件来说，控件升级成组件的自由度更高，可以有很多方式进行组合，这些组合方法不一定是常规的。对于 GUI 来说，规范各种控件、组件的实现方式是一件较难的事情，如果要强制在不同控件、组件上增加不同的属性，对于开发者来说就更难了。但是对于 VUI 来说，GUI 到底长什么样 VUI 是不关心的，所以笔者认为在 VGUI 里用 {} 的方式定义每个最小的交互模块，具体是将以上属性跟控件绑定在一起，还是跟视图（View）绑定在一起可以由开发者自行定义，这样做能降低 GUI 和 VUI 在开发上的耦合度，同时降低对开发者重构代码的要求。

名字：在 5.4.2 节中提到，如果只通过单个控件无法表达完整的信息，那么设计师应该考虑通过组件进行表达，例如单独的滑块只能代表某个数值，需要和标签或图标结合在一起才能让用户知道这个滑块的操作对象是什么。在 VGUI 中，每个独立可用的控件或组件的属性列表里都需要有“name”这个属性，它直接关系到用户和系统知不知道这个控件、组件或指令叫什么。如果遗漏了，即使用户说出按钮上的文案，系统也不知道这条指令指的是哪个交互对象，所以名字对于 VGUI 来说非常重要。

执行范围：该控件/组件只允许在当前界面执行，代表该语音指令是局部指令；允许在其他界面执行，代表该语音指令是全局指令。从执行范围来看，跟控件/组件绑定的全局指令也是局部指令中的一种，所以笔者在前文提过全局指令和局部指令无法做到完美地分离，但是将它们在概念上做区分有利于设计师梳理哪些指令应该是全局响应的、哪些指令不应该是全局响应的。不应该全局响应的指令有上一页/下一页的翻页按钮、收藏、分享等和

上下文强相关的功能，以及可以缩放的图片和地图控件等，这些都应该只允许在当前界面执行。

控件状态和当前状态：开关有开和关两个状态；滑块有最小值、最大值、移动颗粒度及当前数值，不同控件有着不同的状态和属性定义。在前面提到，这些最小的交互模块内部有可能是由不同控件和组件混合而成的，所以要将最小交互模块包含的状态和属性定义清楚，尤其是当前状态，因为当前状态是联系 GUI 和 VUI 的桥梁。

区域内多模交互响应容错：当控件/组件互斥时，交互事件以客户端最后一次操作的时间为准。该属性理解起来有一点烧脑。多个模态之间的合作在时间上可以分为同步合作和异步合作。同步合作是指多个模态同时工作，例如接收信息时会边看边听。异步合作是指多个模态在前后合作，输出信息一般属于异步合作，这时要考虑合作的顺序。以一个开关为例，如果用户用语音指令打开一个开关，但因为网络的原因语音指令迟迟不能下发，这时用户点击了“开关”按钮，那么开关会处于关闭状态吗？不会，因为语音指令迟迟没有响应，所以用户才会将开关打开，因此开关处于开启状态；这时语音指令下发了，请问开关需要回到关闭状态吗？建议不需要，因为用户的意图是打开开关，如果迟来的语音指令重新触发操作，会跟用户的意图产生冲突。所以，在同一个交互对象上进行多通道交互，要以客户端最后一次操作的时间为准，因为服务端下发指令的时间不能代表用户操作的时间。

当组件内部多个控件存在互斥现象时，要以客户端最后一次操作的时间为准。例如，菜单内的选项只能选择 1 个，当前菜单有 3 个选项，分别是选项 1、选项 2 和选项 3，当前焦点在选项 1 上，假设用户先通过语音交互的方式选中选项 2，再通过触控的方式选中选项 3，那么当前菜单的焦点一定是在选项 3 上吗？跟上述例子同理，因为绝大部分的语音交互需要通过云端进行识别和理解，处理过程需要一定的时间，如果从云端返回到客户端的 GUI 操作指令下发时间晚于用户触摸选项 3 的时间，不做任何处理的系统会

将当前菜单的焦点落在选项 2 上，这跟用户的意图和操作不符，因为用户的意图已经发生改变，这时菜单焦点应该落在选项 3 上，如图 5-16 所示。



图 5-16 多通道和多个选项之间的冲突

再举一个例子: 假设当前出现一个弹窗, 在这个弹窗内有一个开关控件, 用户用语音将开关打开, 但是因为网络原因语音指令迟迟不下发, 这时用户点击了弹窗的“关闭”按钮, 然后语音指令下发了, 请问该开关控件是否应该被打开? 在这里笔者建议开关控件仍应处于关闭状态, 因为从用户的视角来看, 用户看到开关仍处于关闭状态后才关闭了弹窗, 他知道该操作没有正常执行, 如果语音指令突然执行, 用户可能会感到困惑, 同时用户无法确认弹窗内是否还会发生什么, 所以笔者不建议晚到的语音指令还要被执行。从系统的角度来看, 刚刚举的三个例子客户端都无法知道云端几时会把指令下发下来, 如果真的要将该条指令顺利执行, 只有两种办法, 第一种就是阻塞其他交互事件, 也就是用户发出语音指令后全部控件不能响应, 只有等指令从云端回来并执行完毕, 控件才能继续响应, 这种方法是完全不可取的。另外一种方法是迟来的指令继续执行, 一次延迟可能还能接受, 如果出现 10 次甚至 100 次延迟, 这时即使系统不紊乱用户可能也懵了, 所以每个模态之间的协同需要以同一个时间作为参照物, 而这个时间是以客户端为准的时间。笔者建议, 如果云端返回的指令时间延迟超过 5 秒, 该指令就可以直接被屏蔽掉, 然后语音回复“当前网络较差, 请稍后再试”。整体效果如图 5-17 所示。



图 5-17 多通道和开关、弹窗之间的冲突

允许交互/不可交互：所有的交互控件/组件都有 Enable 和 Disable 两个状态，本身没有执行成功和执行失败两个状态，常见的 Disable 状态下控件/组件会通过置灰的方式显性告知用户当前状态不可交互。而在语音交互中只有执行成功和执行失败两个状态，没有“该指令不可交互”的说法，所以两者在状态设计上有一定的差异。交互控件/组件在交互时一般有 Default、Hover、Active、Focus4 个状态，每个状态在切换时会有轻微的视觉变化。在用 VUI 控制 GUI 的过程中，用户有可能隔着一米的距离跟交互界面交互，这些轻微的视觉变化对于用户来说并不明显，所以笔者建议在用 VUI 控制 GUI 时，根据当前交互状态提供更显性的动画效果，如图 5-18 所示。



图 5-18 触碰点击按钮和语音控制按钮的区别

成功执行 VUI 响应事件/执行失败 VUI 响应事件：前面提到语音交互中只有执行成功和执行失败两个状态，进入 VUI 响应事件说明系统听懂用户说了什么，执行失败的原因系统是可知，因此建议执行失败不要只告诉用

户“执行失败”，而是将执行失败的原因告诉用户。以调节空调为例，假设空调温度调节范围为 18℃~35℃，当前 25℃，用户说调到 45℃，从结果来看空调无法将温度调节到 45℃，如果只是告诉用户“调节温度执行失败”，用户不看界面永远不知道问题来自哪里。友善的失败反馈应该是“当前空调最高温度 35℃，无法将温度调节至 45℃”，更友好的响应方式可以是“抱歉，无法帮你将空调温度调节至 45℃，已为你将温度调节至最高温度 35℃了”。除用语音播报作为反馈外，VUI 响应事件还可以用不同的音效作为反馈，详情已在 5.5.1 节讲述，这里不再提及。

跳转事件/当前界面执行：界面转场动画效果较明显，即使用户隔着一米的距离或用余光也能看到界面发生跳转，因此笔者建议在这个条件下不需要通过 VUI 进行反馈，如果想保持体验的一致性，可以适当加入音效作为反馈。

至此，已讲解完 VGUI 控件和组件应该如何设计，下面的图 5-19~图 5-23 是笔者梳理的一部分控件和组件，供读者参考。

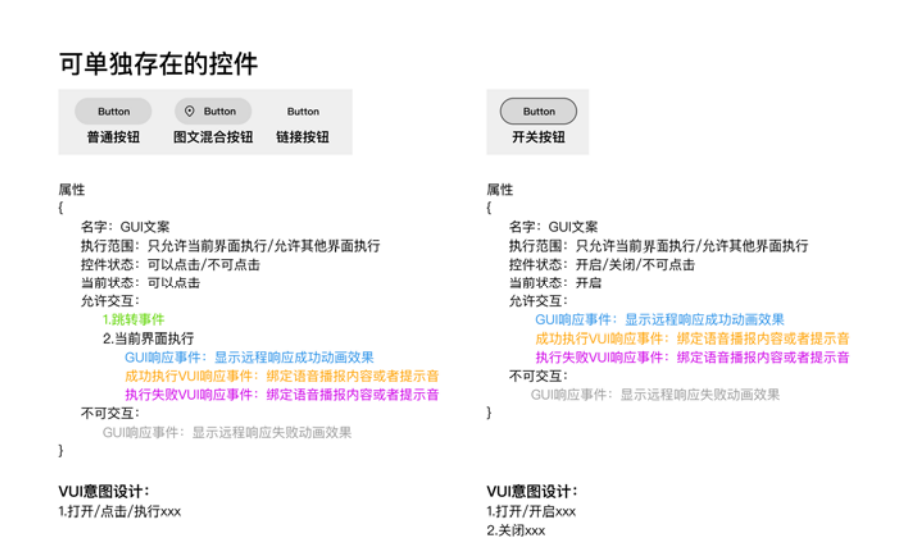


图 5-19 部分可单独存在的控件 1

可单独存在的控件



属性

```
{
  名字: xxx分段控制
  执行范围: 只允许当前界面执行/允许其他界面执行
  控件状态: item A/item B/item C
  当前状态: item A
  区域内多模交互响应容错: 以用户最后一次操作为准
  允许交互:
    GUI响应事件: 显示远程响应成功动画效果
    成功执行VUI响应事件: 绑定语音播报内容或者提示音
}
```

VUI意图设计:

1.打开/点击/执行item X



属性

```
{
  名字: xxx导航栏
  执行范围: 只允许当前界面执行
  控件状态: item A/item B/item C/item D
  当前状态: item A
  区域内多模交互响应容错: 以用户最后一次操作为准
  允许交互:
    GUI响应事件: 显示远程响应成功动画效果
    成功执行VUI响应事件: 绑定语音播报内容或者提示音
}
```

VUI意图设计:

1.打开/进入item X

图 5-20 部分可单独存在的控件 2

组件



属性

```
{
  名字: xxx设置
  执行范围: 只允许当前界面执行/允许其他页面执行
  控件状态: item A/item B/item C
  当前状态: item A
  区域内多模交互响应容错: 以用户最后一次操作为准
  允许交互:
    GUI响应事件: 显示远程响应成功动画效果
    成功执行VUI响应事件: 绑定语音播报内容或者提示音
}
```

VUI意图设计:

1.把xxx设置为yyy



属性

```
{
  名字: xxx开关
  执行范围: 只允许当前页面执行/允许其他页面执行
  控件状态: 开启/关闭/不可点击
  当前状态: 开启
  允许交互:
    GUI响应事件: 显示远程响应成功动画效果
    成功执行VUI响应事件: 绑定语音播报内容或者提示音
    执行失败VUI响应事件: 绑定语音播报内容或者提示音
  不可交互:
    GUI响应事件: 显示远程响应失败动画效果
}
```

VUI意图设计:

1.打开/开启xxx/xxx (字数过长时可以忽略打开/开启)

2.关闭xxx

图 5-21 部分组件 1

组件



属性

```
{
  名字: xxx设置
  执行范围: 只允许当前页面执行
  控件状态: 选中/没选中/不可点击
  当前状态: 选中
  允许交互:
    GUI响应事件: 显示远程响应成功动画效果
    成功执行VUI响应事件: 绑定语音播报内容或者提示音
  不可交互:
    GUI响应事件: 显示远程响应失败动画效果
}
```

VUI意图设计:

- 1.选中xxx
- 2.取消选中xxx



属性

```
{
  名字: xxx设置
  执行范围: 只允许当前页面执行/允许其他页面执行
  控件状态: 下限操作/上限操作/执行颗粒度
  当前状态: yyy
  允许交互:
    GUI响应事件: 显示远程响应成功动画效果
    成功执行VUI响应事件: 绑定语音播报内容或者提示音
    执行失败VUI响应事件: 绑定语音播报内容或者提示音
  不可交互:
    GUI响应事件: 显示远程响应失败动画效果
}
```

VUI意图设计:

- 1.把xxx设置为yyy

图 5-22 部分组件 2

容器



属性:

```
{
  名字: xxx对话框
  执行范围: 只允许当前界面执行
  响应事件: 根据容器内的控件和组件分别执行
}
```

VUI意图设计:

- 1.根据容器内的控件和组件分别设计



属性:

```
{
  名字: xxx列表
  执行范围: 只允许当前界面执行
  响应事件: 根据容器内的控件和组件分别执行
  items多模交互响应容错: 以用户最后一次操作为准
}
```

VUI意图设计:

- 1.根据容器内的控件和组件分别设计
- 2.选中第xxx个

图 5-23 部分容器

5.5.3 手势的融合

除控件/组件可以和语音指令交互外，触控手势也可以和语音指令进行绑定，尤其是以下常用的触控手势。

跟页面滑动相关的手势：当列表中的内容超过一屏时，可以支持“向下滑”“向上滑”等说法，这时匀速向下/向上滑动 30%（本节内容涉及的数值仅作为示意）的列表高度，如果用户说“向下翻页”或“向上翻页”，则可以替用户向下/向上滑动 90%的列表高度。用户可以通过说“回到顶部”“返回最上方”实现返回列表最顶端的功能。由于现在很多内容列表并不支持左右滑动，而且采用了底部上拉无限加载的方式，所以并不支持“向左滑动”“向右滑动”“滑动到页面底部”指令，这时可以统一播报“暂时不支持这项操作”。

跟图片、地图相关的手势：支持“放大到最大”“缩小到最小”的说法；支持“放大一点”“缩小一点”的说法，这时内容放大或缩小 20%；支持“往上/下/左/右移动”的说法，这时内容向上/下/左/右移动 20%；地图支持“放大/缩小到 X 米”的说法，X 可以是 10、20、100 等。

除触控手势外，第 4 章提及的隔空交互手势也可以操控 GUI。本质上语音交互和隔空交互手势都属于远程交互方式，语音交互在 GUI 上的反馈设计包括远程响应动画效果、语音播报及音效反馈都能和隔空交互手势的执行成功、失败反馈设计相结合。以上就是多模交互中 GUI 和 VUI 的融合设计，也是整本书的核心内容。